# 3_2_Ecuacion_de_calor_Dirichlet

May 15, 2026

## 1 Ecuacion de calor con condiciones de Dirichlet (Actividad)

$$\frac{\partial u}{\partial t} + a\frac{\partial u}{\partial x} = D\frac{\partial^2 u}{\partial x^2}, \quad x \in (0,1),\ t \geq 0$$

con $u(0,t) = u(1,t) = 0$ y $u(x,0) = \sin(4\pi x)$.

Karla Rebeca Munguía Romero

Cristobal Medina Meza

Gabriel Eduardo Meléndez Zavala

Gabriel Reynoso Escamilla

Santiago Mora Cruz

Guillermo Villegas Morales

### 1.1 Libraries

[1]:
```
import numpy as np
from numpy.linalg import solve, norm
from numpy import pi, exp, sin, cos
import matplotlib.pyplot as plt
from matplotlib import cm
import matplotlib.animation as animation
import pandas as pd
from pathlib import Path
```

### 1.2 Archivos de solucion exacta (CSV)

[2]:
```
base_dir = Path("./datos")
exact_files = {
    (1.0, 20): "uexmp_1_2_20.csv",
    (1.0, 40): "uexmp_1_2_40.csv",
    (0.01, 20): "uexmp_001_2_20.csv",
    (0.01, 40): "uexmp_001_2_40.csv",
}

def load_exact(D, N):
```

```
    key = (float(D), int(N))
    if key not in exact_files:
        raise KeyError(f"No hay archivos configurados para D={D}, N={N}")
    exact_path = base_dir / exact_files[key]
    if not exact_path.exists():
        raise FileNotFoundError(f"No se encontro: {exact_path}")
    u_exact_grid = np.loadtxt(exact_path, delimiter=",")
    if u_exact_grid.shape[0] != (N + 1):
        u_exact_grid = u_exact_grid.T
    u_exact = u_exact_grid
    return u_exact, u_exact_grid
```

[3]:
```
u_exact, u_exact_grid = load_exact(D=1.0, N=20)
```

[4]:
```
u_exact_grid.shape
```

[4]: (21, 1001)

[5]:
```
u_exact.shape
```

[5]: (21, 1001)

### 1.3 Parametros del problema

[6]:
```
a = 2.0
D_values = [1.0, 0.01]
N_values = [20, 40]

tmin, tmax = 0.0, 1.0
dt = 0.001

times_to_plot = [0.06, 0.1, 1.0]


def f(x):
    return np.sin(4*np.pi*x)
```

### 1.4 Esquemas

Se usan diferencias centradas para $u_x$ y $u_{xx}$.

$$u_i^{k+1} - u_i^k + \alpha\left(u_{i+1}^{k+1} - u_{i-1}^{k+1}\right)$$

$$= s\left(u_{i+1}^{k+1} - 2u_i^{k+1} + u_{i-1}^{k+1}\right)$$

$$-u_i^k = -u_i^{k+1} - \alpha\left(u_{i+1}^{k+1} - u_{i-1}^{k+1}\right)$$

$$+ s\left(u_{i+1}^{k+1} - 2u_i^{k+1} + u_{i-1}^{k+1}\right)$$

$$u_i^k = -u_{i+1}^{k+1}(s+\alpha) - u_i^{k+1}(-2s-1)$$

$$- u_{i-1}^{k+1}(s+\alpha)$$

$$\begin{pmatrix} \alpha-s & 2s+1 & -s-\alpha \end{pmatrix} \begin{pmatrix} u_{-1}^1 \\ u_0^1 \\ u_1^1 \end{pmatrix} = \begin{pmatrix} u_0^0 \end{pmatrix}$$

$$u_i^{k+1} - u_i^k + \alpha\left(u_{i+1}^{k+1} - u_{i-1}^{k+1}\right)$$

$$= s\left(u_{i+1}^{k+1} - 2u_i^{k+1} + u_{i-1}^{k+1}\right)$$

$$-u_i^k = -u_i^{k+1} - \alpha\left(u_{i+1}^{k+1} - u_{i-1}^{k+1}\right)$$

$$+ s\left(u_{i+1}^{k+1} - 2u_i^{k+1} + u_{i-1}^{k+1}\right)$$

$$u_i^k = -u_{i+1}^{k+1}(s+\alpha) - u_i^{k+1}(-2s-1)$$

$$- u_{i-1}^{k+1}(s+\alpha)$$

$$\begin{pmatrix} \alpha - s & 2s+1 & -s-\alpha \end{pmatrix} \begin{pmatrix} u_{-1}^1 \\ u_0^1 \\ u_1^1 \end{pmatrix} = \begin{pmatrix} u_0^0 \end{pmatrix}$$

[7]:
```
def setup_case(D, N):
    xmin, xmax = 0.0, 1.0
    dx = (xmax - xmin) / N
    x = np.linspace(xmin, xmax, N + 1)
    M = int((tmax - tmin) / dt)
    t = np.linspace(tmin, tmax, M + 1)
    s = D * dt / (dx * dx)
    alpha = a * dt / (2 * dx)
    X, T = np.meshgrid(x, t)
    vmin, vmax = -1.0, 1.0
    levels = np.linspace(vmin, vmax, 21)
    return x, t, dx, M, s, alpha, X, T, levels


def forward_euler(D, N):
    x, t, dx, M, s, alpha, X, T, levels = setup_case(D, N)
    # evitar overflow no quitar
    if s > 0.5 or abs(alpha) > 1.0:
        raise ValueError(
            f"Forward Euler unstable for D={D}, N={N}: s={s:.3f}, alpha={alpha:.
 ↪3f}. "
            "Reduce dt or N."
        )
    u = np.zeros((N + 1, M + 1))
    u[:, 0] = f(x)
    for k in range(M):
        u[1:-1, k + 1] = (
            u[1:-1, k]
            - alpha * (u[2:, k] - u[:-2, k])
            + s * (u[2:, k] - 2 * u[1:-1, k] + u[:-2, k])
        )
    return u, x, t, X, T, levels


def backward_euler(D, N):
    x, t, dx, M, s, alpha, X, T, levels = setup_case(D, N)
    u = np.zeros((N + 1, M + 1))
    u[:, 0] = f(x)
    n = N - 1
    be_matrix_left = np.diag((1 + 2*s) * np.ones(n))
    for j in range(1, n - 1):
        be_matrix_left[j, j + 1] = -(alpha + s)
        be_matrix_left[j, j - 1] = (alpha - s)
    be_matrix_left[0, 1] = -(alpha + s)
    be_matrix_left[-1, -2] = (alpha - s)

    for k in range(M):
```

```
        utemp = u[1:N, k]
        utemp = solve(be_matrix_left, utemp)
        u[1:N, k + 1] = utemp
    return u, x, t, X, T, levels

def crank_nicolson(D, N):
    x, t, dx, M, s, alpha, X, T, levels = setup_case(D, N)
    u = np.zeros((N + 1, M + 1))
    u[:, 0] = f(x)
    n = N - 1
    cn_matrix_left = np.diag(2 * (1 + s) * np.ones(n))
    cn_matrix_right = np.diag(2 * (1 - s) * np.ones(n))
    for j in range(1, n - 1):
        cn_matrix_left[j, j + 1] = (alpha - s)
        cn_matrix_left[j, j - 1] = -(alpha + s)
        cn_matrix_right[j, j + 1] = -(alpha - s)
        cn_matrix_right[j, j - 1] = (alpha + s)
    cn_matrix_left[0, 1] = (alpha - s)
    cn_matrix_left[-1, -2] = -(alpha + s)
    cn_matrix_right[0, 1] = -(alpha - s)
    cn_matrix_right[-1, -2] = (alpha + s)

    for k in range(M):
        utemp = u[1:N, k]
        utemp = cn_matrix_right @ utemp
        utemp = solve(cn_matrix_left, utemp)
        u[1:N, k + 1] = utemp
    return u, x, t, X, T, levels


def error_at_times(u, u_exact_grid, t, times):
    errs = {}
    for tt in times:
        idx = int(round(tt / dt))
        e = u[:, idx] - u_exact_grid[:, idx]
        errs[tt] = {
            "L2": norm(e) / np.sqrt(u.shape[0]),
            "Linf": norm(e, ord=np.inf),
            "e": e,
        }
    return errs
```

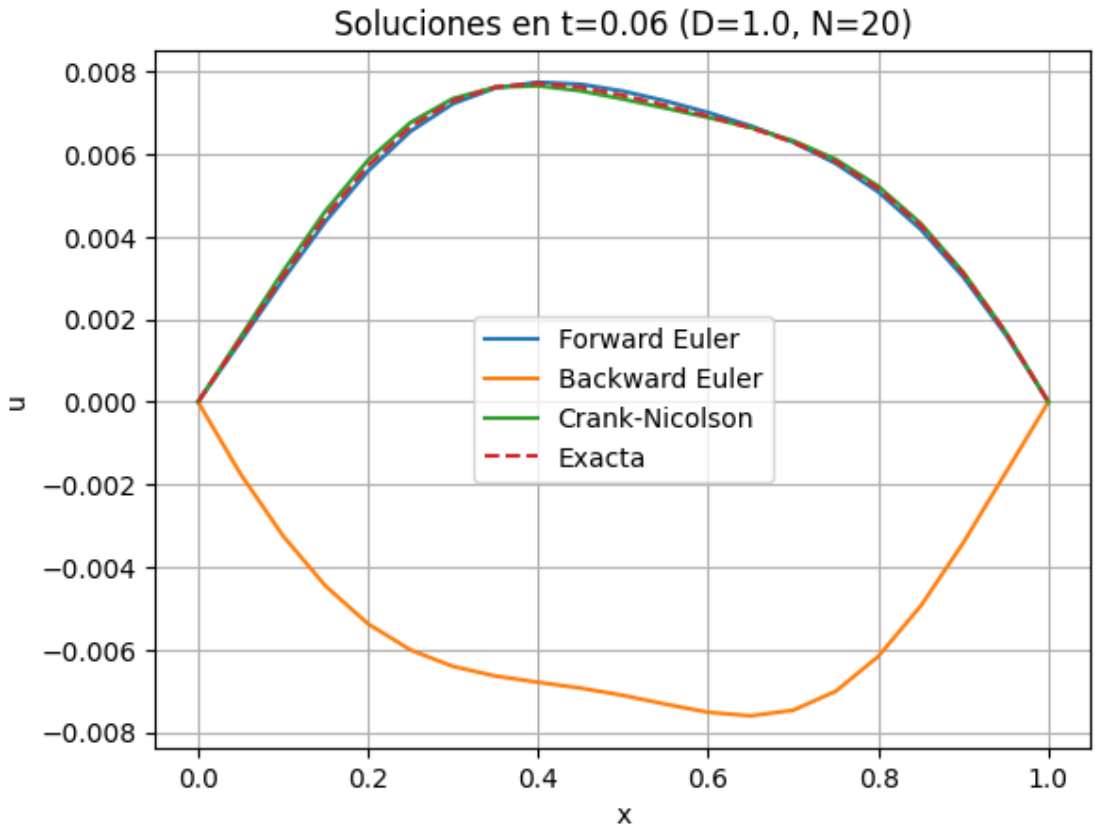
### 1.5 Ejecucion de los casos

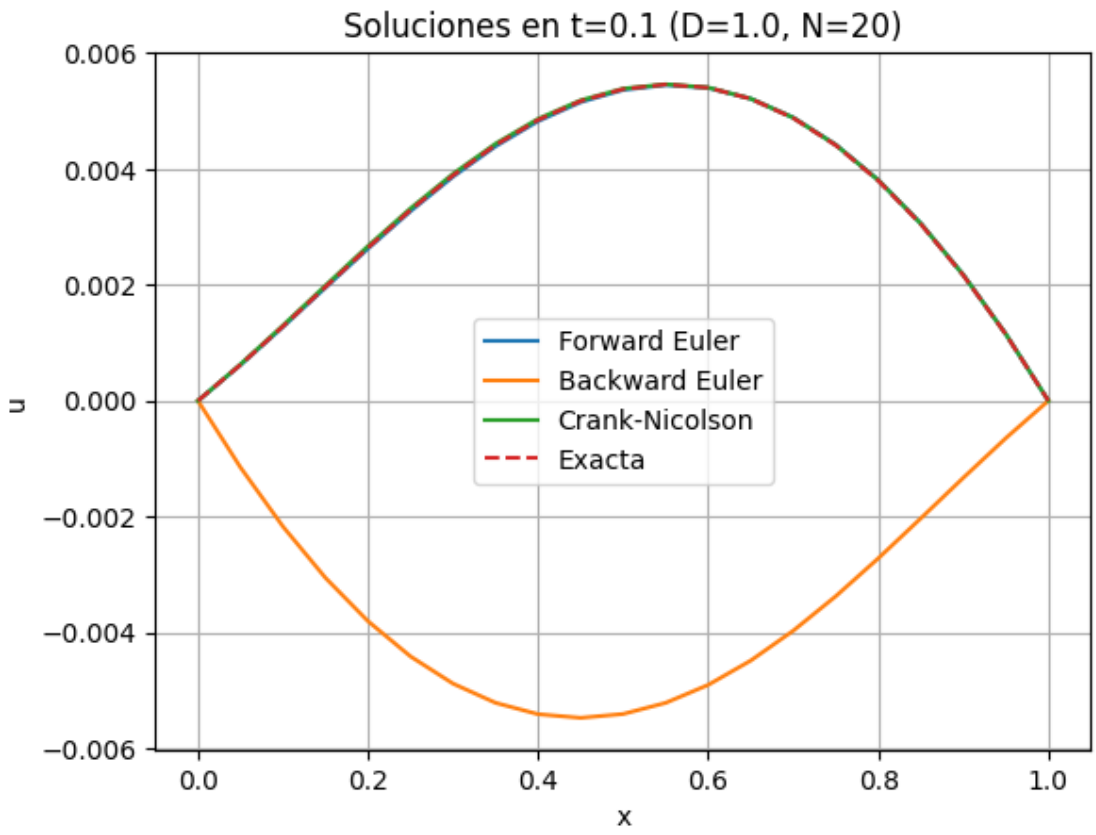
Para cada par $(D, N)$ se cargan los CSV de solucion exacta, se calculan las tres aproximaciones y se comparan en $t = 0.06, 0.1, 1.0$.

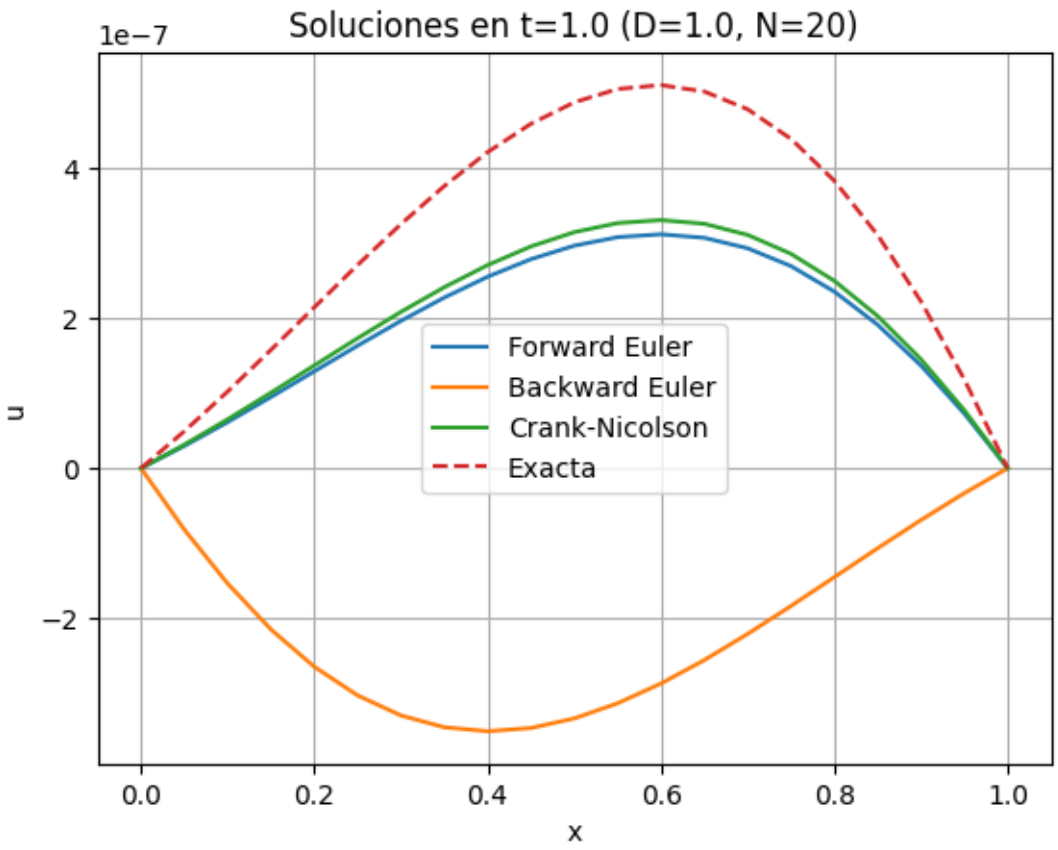
[8]:
```
def run_case(D, N):
    print(f"=== Caso D={D}, N={N} ===")
    u_exact, u_exact_grid = load_exact(D, N)

    x, t, dx, M, s, r, X, T, levels = setup_case(D, N)

    if u_exact_grid.shape != (N+1, M+1):
        raise ValueError(
            f"La solucion exacta discretizada tiene forma {u_exact_grid.shape},␣
↪se esperaba {(N+1, M+1)}"
        )

    try:
        u_fwd, x, t, X, T, levels = forward_euler(D, N)
        u_bwd, x, t, X, T, levels = backward_euler(D, N)
        u_cn, x, t, X, T, levels = crank_nicolson(D, N)
    except ValueError as e:
        print(f"Error: {e}")
        return

    # --- Plots at each required time for each scheme ---
    for tt in times_to_plot:
        idx = int(round(tt / dt))
        fig, ax = plt.subplots()
        ax.plot(x, u_fwd[:, idx],  label="Forward Euler")
        ax.plot(x, u_bwd[:, idx],  label="Backward Euler")
        ax.plot(x, u_cn[:, idx],   label="Crank-Nicolson")
        ax.plot(x, u_exact_grid[:, idx], label="Exacta", linestyle="--")
        ax.set(title=f"Soluciones en t={tt} (D={D}, N={N})", xlabel="x",␣
↪ylabel="u")
        ax.grid()
        ax.legend()
        plt.show()

    # --- Error norms table ---
    errs_fwd = error_at_times(u_fwd, u_exact_grid, t, times_to_plot)
    errs_bwd = error_at_times(u_bwd, u_exact_grid, t, times_to_plot)
    errs_cn  = error_at_times(u_cn,  u_exact_grid, t, times_to_plot)

    rows = []
    for tt in times_to_plot:
        for name, errs in [("Forward Euler", errs_fwd), ("Backward Euler",␣
↪errs_bwd), ("Crank-Nicolson", errs_cn)]:
            rows.append({
                "t": tt,
                "Metodo": name,
                "Error L2": f"{errs[tt]['L2']:.2e}",
```

```
                "Error Linf": f"{errs[tt]['Linf']:.2e}",
            })
    df = pd.DataFrame(rows)
    print(df.to_string(index=False))


for D in D_values:
    for N in N_values:
        run_case(D, N)
```

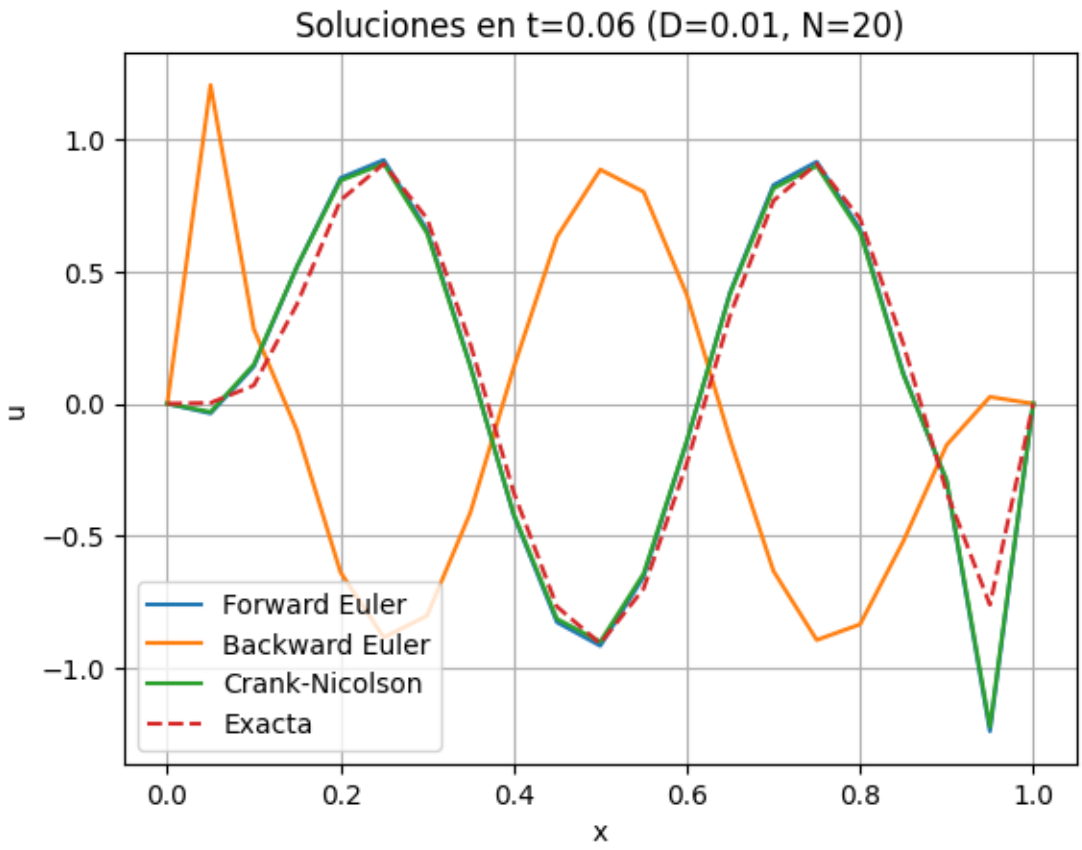
```
=== Caso D=1.0, N=20 ===
```

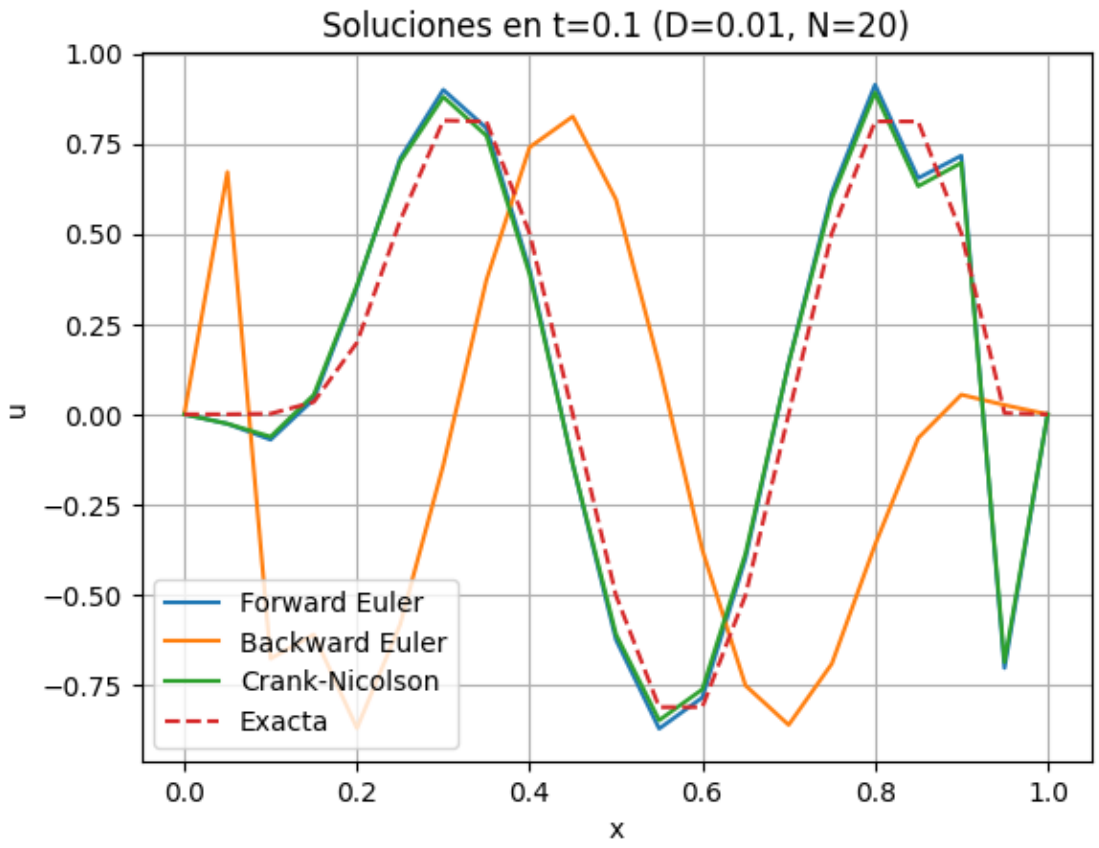


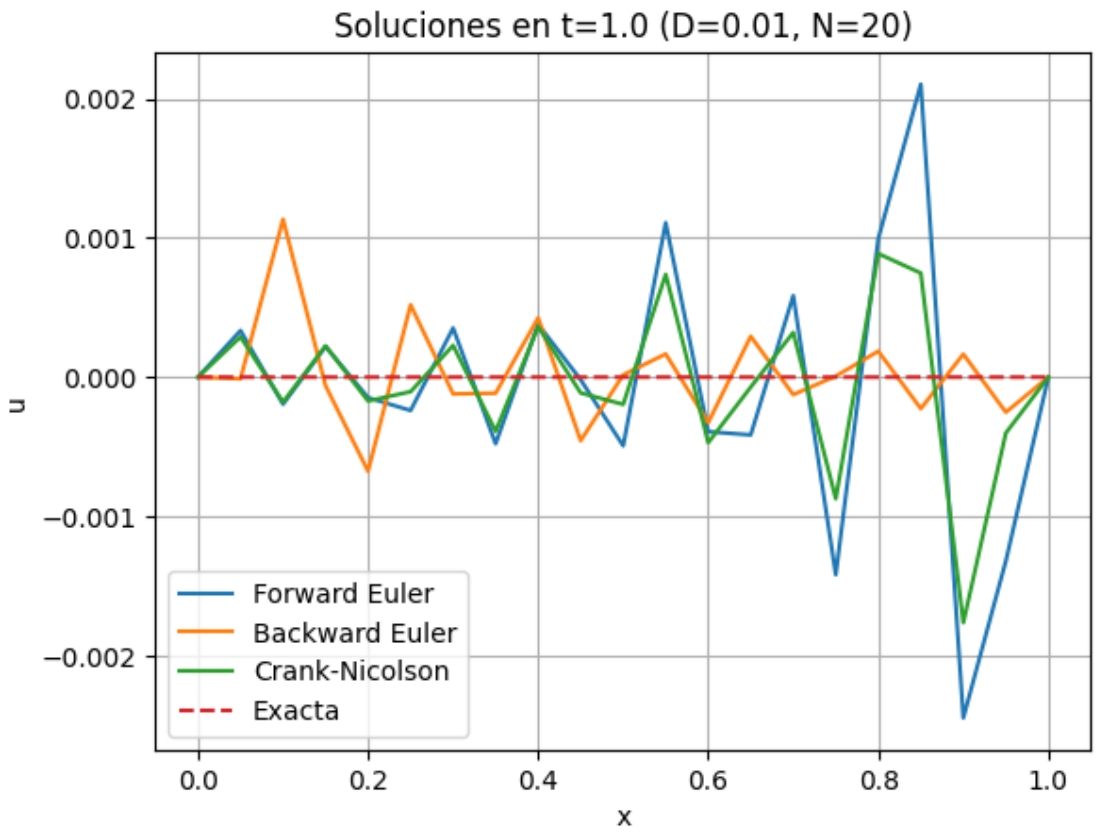

Soluciones en t=0.1 (D=1.0, N=20)
0.006
0.004
0.002
0.000
−0.002
−0.004
−0.006
u
0.0
0.2
0.4
0.6
0.8
1.0
x
Forward Euler
Backward Euler
Crank-Nicolson
Exacta



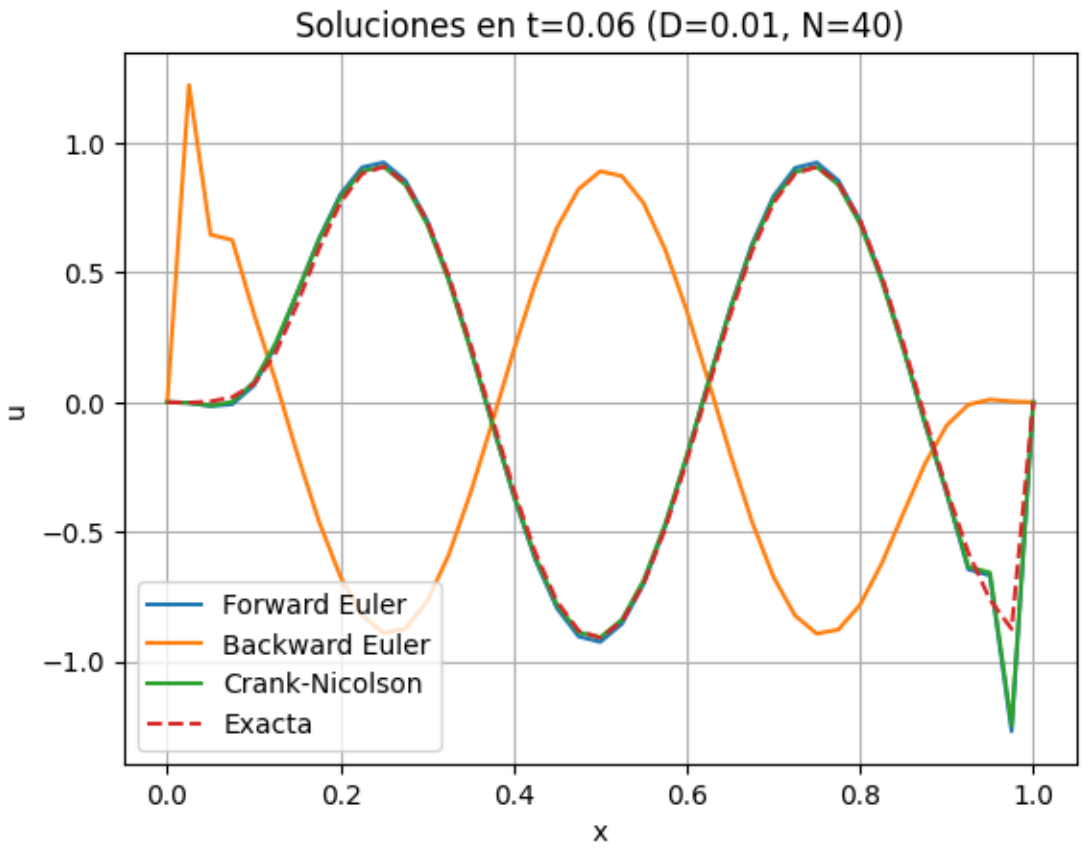

```
   t         Metodo Error L2 Error Linf
0.06  Forward Euler 7.76e-05   1.26e-04
0.06 Backward Euler 1.13e-02   1.45e-02
0.06 Crank-Nicolson 6.97e-05   1.39e-04
0.10  Forward Euler 1.30e-05   2.25e-05
0.10 Backward Euler 7.50e-03   1.08e-02
0.10 Crank-Nicolson 1.48e-05   2.87e-05
1.00  Forward Euler 1.35e-07   1.99e-07
1.00 Backward Euler 5.76e-07   8.21e-07
1.00 Crank-Nicolson 1.23e-07   1.80e-07
=== Caso D=1.0, N=40 ===
Error: Forward Euler unstable for D=1.0, N=40: s=1.600, alpha=0.040. Reduce dt
or N.
=== Caso D=0.01, N=20 ===
```

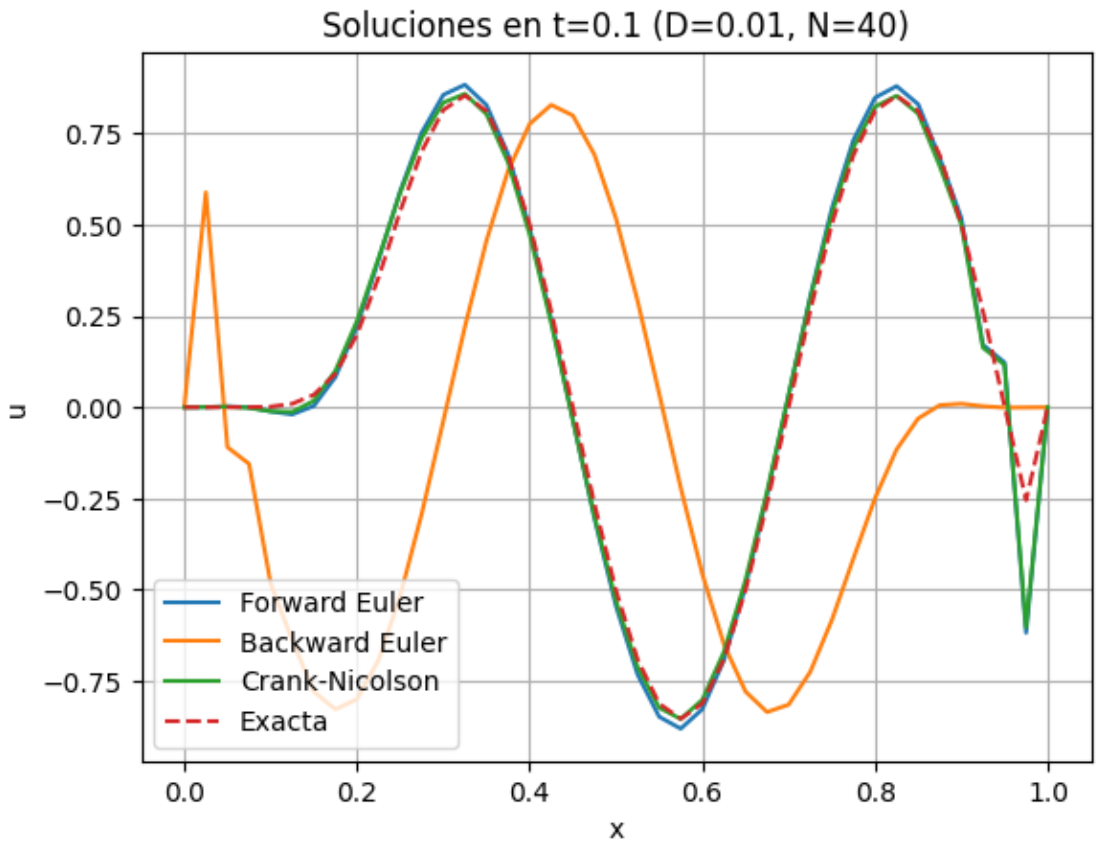

Soluciones en t=0.06 (D=0.01, N=20)
1.0
0.5
0.0
−0.5
−1.0
u
0.0
0.2
0.4
0.6
0.8
1.0
x
Forward Euler
Backward Euler
Crank-Nicolson
Exacta

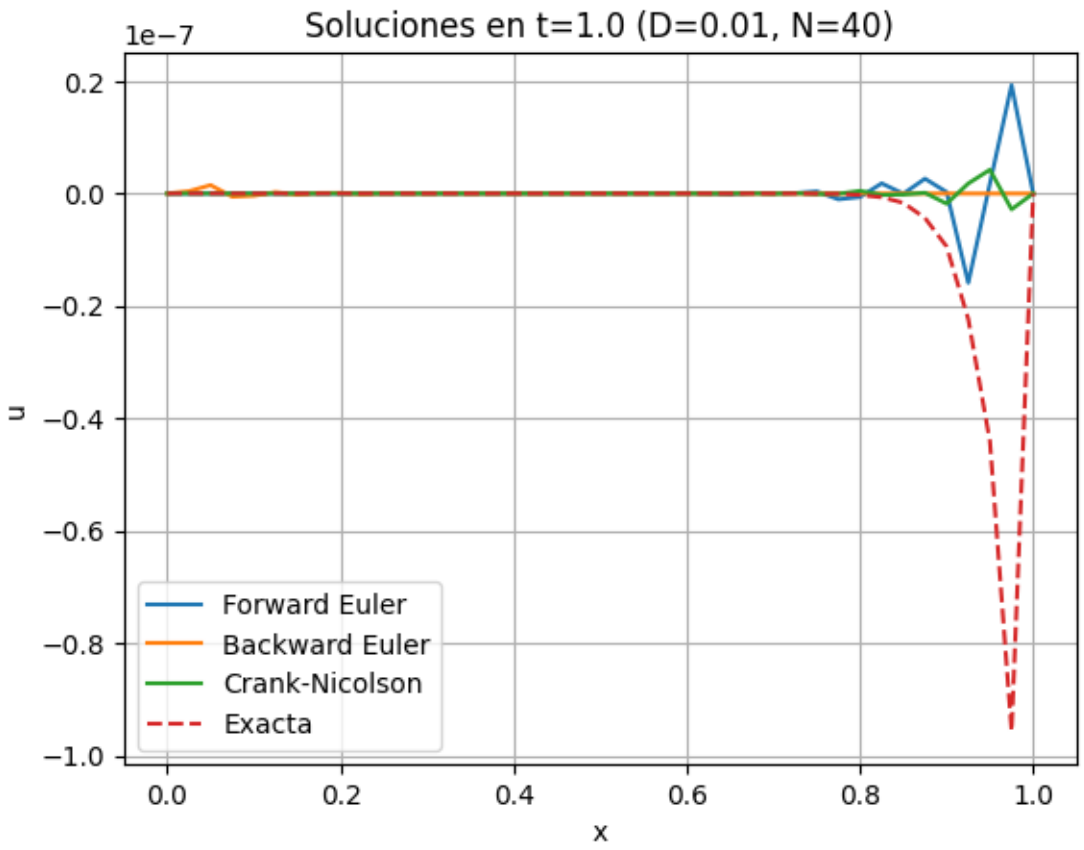

Soluciones en t=0.1 (D=0.01, N=20)
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
u
0.0
0.2
0.4
0.6
0.8
1.0
x
Forward Euler
Backward Euler
Crank-Nicolson
Exacta




```
   t         Metodo Error L2 Error Linf
0.06  Forward Euler 1.23e-01   4.78e-01
0.06 Backward Euler 1.13e+00   1.80e+00
0.06 Crank-Nicolson 1.20e-01   4.61e-01
0.10  Forward Euler 1.88e-01   7.07e-01
0.10 Backward Euler 7.70e-01   1.19e+00
0.10 Crank-Nicolson 1.83e-01   6.90e-01
1.00  Forward Euler 9.28e-04   2.45e-03
1.00 Backward Euler 3.68e-04   1.13e-03
1.00 Crank-Nicolson 5.71e-04   1.76e-03
=== Caso D=0.01, N=40 ===
```


Soluciones en t=0.06 (D=0.01, N=40)
1.0
0.5
0.0
−0.5
−1.0
u
0.0
0.2
0.4
0.6
0.8
1.0
x
Forward Euler
Backward Euler
Crank-Nicolson
Exacta


Soluciones en t=0.1 (D=0.01, N=40)
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
u
0.0
0.2
0.4
0.6
0.8
1.0
x
Forward Euler
Backward Euler
Crank-Nicolson
Exacta




| t | Metodo | Error L2 | Error Linf |
|---|---|---|---|
| 0.06 | Forward Euler | 6.70e-02 | 3.94e-01 |
| 0.06 | Backward Euler | 1.14e+00 | 1.80e+00 |
| 0.06 | Crank-Nicolson | 6.33e-02 | 3.71e-01 |
| 0.10 | Forward Euler | 6.75e-02 | 3.63e-01 |
| 0.10 | Backward Euler | 7.20e-01 | 1.11e+00 |
| 0.10 | Crank-Nicolson | 6.38e-02 | 3.48e-01 |
| 1.00 | Forward Euler | 1.94e-08 | 1.15e-07 |
| 1.00 | Backward Euler | 1.69e-08 | 9.56e-08 |
| 1.00 | Crank-Nicolson | 1.68e-08 | 9.28e-08 |